Experiment No. 2

1. Aim

To implement **parallel searching** and **parallel sorting** algorithms using **OpenMP multi-threading**, and compare their execution with sequential algorithms.

2. Course Outcomes (COs) Addressed

CO No.	Course Outcome
CO1	Apply OpenMP constructs to develop parallel programs.
CO2	Implement parallel algorithms using multiple threads.
CO5	Analyze scalability and performance of parallel programs.

3. Program Outcomes (POs) Mapped

PO	Description	Level
PO1	Engineering Knowledge	3
PO2	Problem Analysis	2
PO3	Design Solutions	2
PO5	Modern Tool Usage	3

4. Learning Objectives

After completing this experiment, students will be able to:

- Understand parallel searching algorithms.
- Understand parallel sorting algorithms.
- Develop OpenMP programs using multiple threads.
- Apply loop parallelization.
- Compare sequential and parallel execution.
- Analyze performance improvement using parallel computing.

5. Theory

5.1 Introduction

Searching and sorting are two fundamental operations used in almost every computing application.
When datasets become very large, sequential algorithms require considerable execution time.
Parallel computing distributes work among multiple processors or threads, thereby improving performance.
Using **OpenMP**, searching and sorting operations can execute simultaneously on different portions of the data.

6. Parallel Searching

Definition

Parallel searching divides the data among multiple threads.
Each thread searches its assigned portion independently.
If any thread finds the required element, it reports the position.

Example

Array
10 20 30 40 50 60 70 80
Target = **70**
Using four threads

Thread	Elements
T0	10 20

T1	30 40
T2	50 60
T3	70 80

Thread 3 finds the element.

Advantages

- Faster searching
- Better CPU utilization
- Efficient for large datasets
- Scalable

7. Parallel Sorting

Definition

Parallel sorting divides the array into smaller portions.
Each thread sorts one portion independently.
Finally, sorted portions are merged into a single sorted array.

Example

Original Array
70 10 80 20 60 30 50 40
Thread 1
70 10 80 20
↓
Sorted
10 20 70 80
Thread 2
60 30 50 40
↓
Sorted
30 40 50 60
↓
Merged
10 20 30 40 50 60 70 80

8. Algorithms Selected

This experiment implements:

Searching Algorithm

- Parallel Linear Search

Sorting Algorithm

- Parallel Bubble Sort (Odd-Even Transposition Sort)

PART-A
Program 1 – Parallel Linear Search Using OpenMP

Algorithm

1. Start.
2. Read array elements.
3. Read the key element.
4. Set the number of OpenMP threads.
5. Divide the array among threads.
6. Each thread searches its assigned elements.
7. If the key is found, store its index.
8. Display the search result.
9. Stop.

Program

```
#include<stdio.h>
#include<omp.h>

int main()
{
   int arr[8]={10,20,30,40,50,60,70,80};
   int key=70;
   int found=-1;

   omp_set_num_threads(4);

   #pragma omp parallel for
   for(int i=0;i<8;i++)
   {
      if(arr[i]==key)
      {
         #pragma omp critical
         {
            found=i;
            printf("Thread %d found %d at index %d\n",
                  omp_get_thread_num(),key,i);
         }
      }
   }

   if(found!=-1)
      printf("\nElement Found at Index %d\n",found);
   else
      printf("\nElement Not Found\n");

   return 0;
}
```

Sample Output

```
Thread 3 found 70 at index 6
```

Element Found at Index 6

Program Explanation

- Four threads are created.
- Each thread searches a portion of the array.
- When the element is found, the critical section safely updates the shared variable found.
- The program reports the index of the searched element.

PART-B

Program 2 – Parallel Bubble Sort (Odd-Even Transposition Sort)

Algorithm

1. Start.
2. Read the array.
3. Create OpenMP threads.
4. Perform odd-even comparison phases.
5. Swap adjacent elements where necessary.
6. Repeat until the array is sorted.
7. Display the sorted array.
8. Stop.

Program

```
#include<stdio.h>
#include<omp.h>

int main()
{
   int arr[8]={70,10,80,20,60,30,50,40};

   int n=8;

   omp_set_num_threads(4);

   for(int phase=0;phase<n;phase++)
   {
      if(phase%2==0)
      {
         #pragma omp parallel for
         for(int i=1;i<n;i+=2)
         {
            if(arr[i-1]>arr[i])
            {
               int temp=arr[i];
               arr[i]=arr[i-1];
               arr[i-1]=temp;
            }
         }
      }
```

```
        else
        {
            #pragma omp parallel for
            for(int i=1;i<n-1;i+=2)
            {
                if(arr[i]>arr[i+1])
                {
                    int temp=arr[i];
                    arr[i]=arr[i+1];
                    arr[i+1]=temp;
                }
            }
        }
    }

    printf("Sorted Array:\n");

    for(int i=0;i<n;i++)
        printf("%d ",arr[i]);

    return 0;
}
```

Sample Output

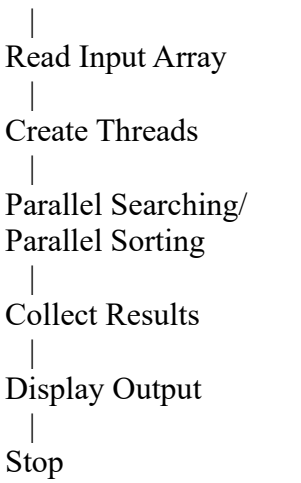
Sorted Array

10 20 30 40 50 60 70 80

Program Explanation

- The algorithm performs multiple odd and even comparison phases.
- Independent comparisons within each phase execute in parallel.
- After all phases, the array becomes fully sorted.

9. Flowchart



10. Execution Steps in Ubuntu Linux

Step 1

Open Terminal
Ctrl + Alt + T

Step 2

Create Program
nano search.c
or
nano sort.c

Step 3

Compile
gcc -fopenmp search.c -o search
gcc -fopenmp sort.c -o sort

Step 4

Execute
./search
./sort

11. Expected Output

Search

Thread 3 found 70 at index 6

Element Found at Index 6

Sort

Sorted Array

10 20 30 40 50 60 70 80

12. Time Complexity

Algorithm	Sequential	Parallel (Approx.)
Linear Search	O(n)	O(n/p)
Odd-Even Bubble Sort	$O(n^2)$	$O(n^2/p)$ (phase-wise parallelism)

Where **n** is the number of elements and **p** is the number of threads.

13. Applications

- Database Management Systems
- Search Engines
- Artificial Intelligence
- Machine Learning
- Image Processing
- Scientific Computing
- Data Analytics
- Cloud Computing
- High Performance Computing
- Operating Systems

14. Advantages

- Reduced execution time
- Better processor utilization
- Improved scalability
- Increased throughput
- Suitable for multi-core processors

15. Limitations

- Thread creation overhead
- Synchronization overhead
- Communication cost
- Load imbalance for irregular data
- Performance gains are significant mainly for large datasets

16. Result

The implementation of **Parallel Linear Search** and **Parallel Bubble Sort (Odd-Even Transposition Sort)** using OpenMP was executed successfully. The experiment demonstrated how multiple threads can work concurrently to improve the performance of searching and sorting operations on shared-memory systems.